8-BIT BREADBOARD COMPUTER

Project Documentation

TEAM MEMBERS

Jana · Amr · Abdelrahman · Monica · Hamza

Digital Logic & Computer Architecture

Faculty of Computer Science | 2025-2026

Inspired by Ben Eater's 8-Bit Breadboard Computer Series

Member	Module Covered	Videos
Jana	Clock / Counter / JK Flip-Flop	2-5, 24-29
Amr	Latches / Registers	6-13
Abdelrahman	ALU (Arithmetic Logic Unit)	14-18
Monica	RAM (Memory Module)	19-23
Hamza	Display / Control Logic / EEPROM	30-44

Table of Contents

1. Abstract

This project documents the design, construction, and testing of a fully functional 8-bit breadboard computer built from the 74-series TTL logic and High-speed CMOS integrated circuits. Inspired by Ben Eater's educational video series, the system implements a complete von Neumann architecture including an adjustable clock, registers, an arithmetic logic unit (ALU), random access memory (RAM), a program counter, control logic driven by EEPROM microcode, and a 7-segment decimal display output.

The computer is capable of executing a custom instruction set including load, store, add, subtract, jump, and output operations. The project was divided among five team members, each responsible for a specific hardware module, with integration and testing performed as a group. The final system successfully runs programs such as calculating the Fibonacci sequence, demonstrating correct instruction fetch, decode, and execute cycles across all modules.

> Key Achievement: A fully programmable 8-bit CPU assembled from logic gates on breadboards, capable of executing stored programs with conditional branching.

2. Objectives

The primary goal of this project was to bridge the gap between theoretical computer architecture and physical hardware implementation. The team set out to achieve the following:

- Understand the internal structure of a CPU by physically constructing one from individual logic gates and flip-flop ICs.
- Implement all major subsystems of a von Neumann architecture: clock, registers, ALU, RAM, program counter, and control unit.
- Design and program EEPROM-based microcode to drive the control logic for each instruction.
- Develop and execute test programs on the completed machine, including arithmetic operations and iterative sequences.
- Practice engineering teamwork by dividing a complex system into modules with clear interfaces.
- Document the design and build process to a professional university standard.

3. System Overview

The computer follows the classic SAP-1 (Simple As Possible) architecture. All modules share an 8-bit data bus. The control unit generates a set of control signals each clock cycle that dictate which modules drive or read the bus.

Data Bus Architecture

The central 8-bit bus connects all major modules. At any given clock step, at most one module drives the bus (output enable) and one or more modules read from it. Tri-state buffers on each module ensure that inactive modules remain electrically disconnected from the bus, preventing contention.

Module	Function	Responsible Member
Clock	Generates the master clock signal (manual/auto modes)	Jana
A & B Registers	General-purpose 8-bit data registers	Amr
ALU	Performs addition and subtraction on A and B registers	Abdelrahman
RAM (16 bytes)	Stores program instructions and data	Monica
Program Counter	Tracks current instruction address (4-bit)	Jana
Instruction Reg.	Holds current opcode and operand	Amr
Control Logic	Decodes instructions via EEPROM microcode	Hamza
Display Output	Converts binary result to decimal on 7-segment display	Hamza

4. Theory Background

4.1 Binary Number System

All data in the computer is represented in base-2 (binary). An 8-bit word can represent unsigned values from 0 to 255, or signed values from -128 to 127 using two's complement notation. Arithmetic overflow is tracked by the carry flag register.

4.2 Logic Gates, TTL & High-speed CMOS ICs

The computer is built using a combination of 74LS-series (low-power Schottky TTL) and 74HC-series (High-speed CMOS) integrated circuits. Key IC families used include:

- 7400 / 7404 - NAND and NOT gates for glue logic
- 7486 - XOR gates used in the ALU subtractor
- 74283 - 4-bit binary full adder (two used for 8-bit addition)
- 74173 - 4-bit D-type registers with tri-state outputs
- 74245 - 8-bit bus transceiver for bus interface
- 74161 - 4-bit synchronous binary counter (program counter)
- 74189 - 16x4 bit static RAM

4.3 Flip-Flops & Registers

A D-type flip-flop captures the value on its data input at the rising edge of the clock and holds it until the next clock edge. Registers are built from multiple flip-flops in parallel. The A and B registers each hold 8 bits and can be selectively loaded from the bus or output to the bus under control signal direction.

4.4 Tri-State Bus

A tri-state buffer has three output states: logic HIGH, logic LOW, and high-impedance (Hi-Z). When in Hi-Z mode, the buffer's output is effectively disconnected. This allows multiple modules to share a single bus without conflict, only the module with its output-enable active will drive the bus at any time.

4.5 Microcode & EEPROM

Each instruction is broken into a sequence of micro-operations, each lasting one clock cycle. The EEPROM stores a lookup table indexed by the current instruction opcode and the current step counter value. Each table entry is a byte where each bit corresponds to one control signal (e.g., register load enable, ALU subtract mode, RAM output enable). This approach uses the EEPROM to replace combinational logic, significantly simplifying the hardware design. What would otherwise require dozens of logic gate ICs to decode each instruction is reduced to a single chip lookup, allowing complex instructions to be implemented in firmware rather than in hardware wiring.

4.6 Two's Complement Subtraction

Subtraction is performed by the ALU by inverting the bits of the B register (using XOR gates with a subtract control line) and adding 1 via the carry-in of the adder, effectively computing A + (~B) + 1 = A - B. This eliminates the need for a separate subtractor circuit.

5. Module Descriptions

5.1 Clock Module - Jana

The clock module provides the master timing signal for the entire computer. It supports two operating modes:

- Automatic mode - a 555-timer astable circuit generates a continuous square wave. The frequency is adjustable via a potentiometer, ranging from approximately 1 Hz to several hundred Hz.
- Manual (single-step) mode - a debounced push button generates exactly one clock pulse per press, allowing the user to step through instructions one cycle at a time for debugging.

A monostable 555 debounce circuit is used on the manual button to eliminate contact bounce, which would otherwise produce multiple spurious clock edges. A third 555 IC configured as a monostable is used to debounce the mode-select switch itself, ensuring that toggling between automatic and manual modes does not introduce spurious clock pulses during the transition. The output of the clock module is distributed to all synchronous modules via a dedicated clock rail on the bus board.

> Key IC: NE555 timer (x3) - one astable, two monostable for debouncing.

5.2 Registers Module - Amr

The register module implements two 8-bit general-purpose registers, designated A and B, plus the Instruction Register (IR).

- Register A is the primary accumulator. It feeds one input of the ALU at all times and can receive data from the bus.
- Register B feeds the second ALU input. It is write-only from the bus; its value is not directly output to the bus.
- The Instruction Register holds the currently executing instruction. The upper 4 bits (opcode) are sent to the control logic; the lower 4 bits (operand/address) can be placed on the bus for memory addressing.

All registers use 74173 quad D-type flip-flop ICs with built-in tri-state output buffers, directly controlled by the output-enable control signals from the control unit.

5.3 ALU Module - Abdelrahman

The Arithmetic Logic Unit performs 8-bit addition and subtraction. It is built from two 74283 4-bit binary adders cascaded to form an 8-bit adder. Subtraction is achieved using the XOR-based two's complement trick described in Section 4.6.

- When the SUB control line is LOW: the ALU computes A + B.
- When the SUB control line is HIGH: B inputs are inverted by 7486 XOR gates, and carry-in is set HIGH, computing A - B.

The ALU result is always available at its output, but only drives the bus when the ALU output-enable signal is asserted. A flag register captures the carry-out and zero condition of the result on the appropriate clock edge, enabling conditional jump instructions.

5.4 RAM Module - Monica

The memory module provides 16 bytes of addressable storage using a 74189 16x4 static RAM (two ICs combined for 8-bit data width). The 4-bit address is held in the Memory Address Register (MAR), which is loaded from the bus.

- Program mode - DIP switches allow manual entry of a program byte-by-byte before execution.
- Run mode - the computer reads instructions and data from RAM automatically during program execution.

The small 16-byte address space is a fundamental constraint of the 4-bit address bus. All programs must fit within 16 instructions and data bytes combined.

> Note: The 16-byte RAM limit constrains program length. Fibonacci output overflows 8 bits at 233 and wraps around - visually interesting on the display.

5.5 Control Logic & Display - Hamza

The control unit is the brain of the computer. It is implemented using two 28C16 EEPROMs that store the microcode lookup table. The address input to the EEPROMs is formed by concatenating the instruction opcode (4 bits), the current micro-step counter value (3 bits), and the flag register bits (carry and zero).

- The EEPROM output byte drives 16 active-low control signals: HLT, MI, RI, RO, IO, II, AI, AO, EO, SU, BI, OI, CE, CO, J, FI.
- A 3-bit step counter (74161) cycles through up to 8 micro-steps per instruction, resetting at the end of each instruction.

The output display module converts the binary value in the A register to a human-readable decimal number on four 7-segment LED displays, supporting both unsigned mode (0 to 255) and signed two's complement mode (-128 to 127).

Rather than dedicating one EEPROM per display digit, all four lookup tables are stored in a single EEPROM by partitioning the address space, address lines A9 and A10 select between the ones, tens, hundreds, and sign digit tables, while address line A11 switches between unsigned and two's complement mode.

A 555 timer rapidly cycles through the four digits by driving a 7476 configured as a 2-bit counter. The counter output feeds a 74139 decoder which activates each 7-segment display in sequence, simultaneously selecting the corresponding lookup table in the EEPROM. The cycling happens fast enough that all four digits appear continuously lit to the human eye.

The EEPROMs were programmed using an Arduino UNO acting as a programmer, with custom sketches written in C++ to output the microcode table and 7-segment patterns.

6. Instruction Set

The computer implements a simple 11-instruction set architecture. Each instruction is one byte: the upper 4 bits are the opcode and the lower 4 bits are the operand (usually a RAM address).

Opcode (binary)	Mnemonic	Operation	Description
0000	NOP	-	No operation
0001	LDA	A ← M[x]	Load A register from RAM address x
0010	ADD	A ← A+M[x]	Add RAM[x] to A, store in A
0011	SUB	A ← A-M[x]	Subtract RAM[x] from A, store in A
0100	STA	M[x] ← A	Store A register to RAM address x
0101	LDI	A ← x	Load immediate value x into A
0110	JMP	PC ← x	Unconditional jump to address x
0111	JC	PC ← x if C	Jump to address x if carry flag set
1000	JZ	PC ← x if Z	Jump to address x if zero flag set
1110	OUT	Display ← A	Output A register to 7-segment display
1111	HLT	-	Halt the clock

7. Components Used

Component	Part Number	Quantity	Purpose
Breadboard 830 Point	MB102	14	Main build platform
Timer IC	NE555	4	Clock oscillator and debounce circuits
Quad 2-Input NAND Gate	7400	2	Glue logic
Quad 2-Input NOR Gate	7402	1	Glue logic
Hex Inverter NOT Gate	7404	5	Signal inversion
Quad 2-Input AND Gate	7408	3	Glue logic
Quad 2-Input OR Gate	7432	1	Glue logic
Dual J-K Flip-Flop	7476	2	Display digit counter
Quad XOR Gate	7486	2	ALU subtraction inversion
3-to-8 Line Decoder	74138	1	Address decoding
2-to-4 Line Decoder	74139	1	Display digit select
Quad 2-to-1 Line Multiplexer	74157	4	Data selection on bus
4-Bit Synchronous Counter	74161	2	Program counter and step counter
4-Bit D-Type Register	74173	8	A, B, IR, MAR, flags registers
64-bit RAM Tri-state	74189	2	16-byte program and data memory
Octal Bus Transceiver	74245	6	Bus interface for all modules
Octal D-Type Flip-Flop	74273	1	Output register
4-Bit Binary Adder	74283	2	ALU addition
Carbon Resistor 10KΩ	—	10	Pull-up/pull-down logic
Carbon Resistor 1KΩ	—	10	LED current limiting
Carbon Resistor 100KΩ	—	10	Timing resistor for 555 circuits
Carbon Resistor 470Ω	—	30	LED current limiting
Carbon Resistor 1MΩ	—	10	Timing resistor for 555 circuits
Potentiometer 1MΩ	—	1	Clock speed adjustment
Ceramic Capacitor 10nF	—	10	Timing capacitor for 555 circuits
Ceramic Capacitor 100nF	—	60	High-frequency decoupling on all ICs
Electrolytic Capacitor 1µF	—	44	Mid-frequency decoupling on all ICs
Electrolytic Capacitor 10µF	—	40	Power rail stabilization
Electrolytic Capacitor 1000µF	—	1	Bulk power supply filtering

Component	Part Number	Quantity	Purpose
DIP Switch 8-Way	—	1	RAM program data input
DIP Switch 4-Way	—	1	RAM address input
LED 5mm Red	—	50	Bus and signal indicators
LED 5mm Yellow	—	10	Signal indicators
LED 5mm Green	—	15	Signal indicators
LED 5mm Blue	—	25	Signal indicators
Push Button 6mm	—	2	Clock manual step and reset
5V 2A Power Adapter	—	1	System power supply
Female DC Jack Adapter	—	1	Power connector to breadboard
Clear Storage Box 10 Grid	—	4	Component organization
26 AWG Solid Core wire	—	—	Connecting everything

8. Construction Process

Construction followed a staged approach, testing each module independently before integration:

1. Power rails tested: 5V and GND distributed across all breadboards. Voltage measured at multiple points. Decoupling capacitors placed at every IC power pin.
2. Clock module built and verified: LED indicator used to confirm correct output in both auto and manual modes.
3. Registers built and tested: A, B, and IR registers loaded and read back via manual bus manipulation.
4. ALU constructed: Addition and subtraction verified with known inputs before connecting to registers.
5. RAM module assembled: Program loaded via DIP switches, read back over bus to verify correct data.
6. Program Counter wired: Counting and jump functionality verified in isolation.
7. Control EEPROM programmed: Microcode table written using Arduino programmer. Contents verified by reading the EEPROM back and manually checking the stored values against the expected microcode table.
8. Display EEPROM programmed: 7-segment patterns verified manually for all unsigned values 0-255 and signed two's complement values -128 to 127.
9. Full integration: All modules connected via shared 8-bit bus. Test programs loaded and executed.

9. Testing & Verification

9.1 Module-Level Testing

Module	Test Method	Pass Criteria
Clock	LED indicator as output	Correct square wave in auto mode, single pulse per button press in manual mode
Registers	Manual bus drive	Correct value stored and output
ALU	Known inputs (e.g. 5+3, 10-4)	Correct sum/difference on bus
RAM	Program loaded, byte-by-byte read-back	All 16 addresses read correctly
PC	Clock pulses counted, JMP tested	Increments and jumps correctly
Control	Single-step through a micro-step	Correct signals per micro-step
Display	Binary values input manually via DIP switches	Correct decimal shown on display

9.2 Integration Tests

- LDA / STA round-trip: value loaded from RAM, modified, stored back, re-read - verified correct.
- ADD instruction: 14 + 28 = 42 displayed on 7-segment - correct.
- SUB instruction: 100 - 37 = 63 displayed - correct.
- JMP instruction: infinite counter loop ran continuously without halting - correct.
- JZ / JC conditional jumps: tested with boundary values (0, 255) - flags set and jumps triggered correctly.
- Fibonacci program: sequence 1, 1, 2, 3, 5, 8, 13, 21, 34... displayed correctly until 8-bit overflow.

10. Problems Encountered & Solutions

Problem	Root Cause	Solution
Random glitches or resetting in some ICs	No decoupling capacitors near IC power pins	Added 100nF ceramic capacitors for high-frequency decoupling and 1µF electrolytic capacitors for mid-frequency decoupling in parallel between the VCC and GND pins of all ICs.
Random outputs in some ICs	Some unused input pins were left floating	Tied all unused pins either HIGH or LOW depending on the logic through a 1kΩ resistor
Clock signal not working correctly	Clock LED used without a resistor	Added a current limiting resistor to all LEDs
Display shows garbage	Wrong code on EEPROM	EEPROM write failures resolved by reading back and verifying contents with the Arduino programmer, then adjusting the write pin timing until reliable programming was achieved.
Inconsistent voltage readings across breadboards	Cheap breadboard contacts have higher resistance causing voltage drop along the power rails	Added 10µF electrolytic capacitors to each power rail to stabilize voltage and reduce drops and spikes

11. Results

The completed computer successfully demonstrated all core CPU functions:

- Executed all 11 instructions in the custom instruction set without errors.
- Correctly performed 8-bit addition and subtraction including carry/borrow detection.
- RAM read and write operations verified across all 16 addressable locations.
- Conditional branching (JZ, JC) operated correctly based on ALU flag output.
- Fibonacci sequence program ran continuously, producing correct values up to the 8-bit overflow point.
- Decimal display correctly rendered all values 0-255, including negative two's complement representation -128-127.
- Clock speed tunable from below 1 Hz (for single-step debugging) up to approximately 500 Hz.

> The Fibonacci program - 7 instructions, 2 data bytes - fit entirely within the 16-byte RAM and ran indefinitely, validating the complete instruction execution pipeline.

12. Reflection & What We Learned

- A modern CPU, despite its complexity, is reducible to simple building blocks: flip-flops, adders, and multiplexers.
- Decoupling capacitors are not optional, digital ICs draw spike currents on switching and require local charge storage.
- Microcode is a powerful abstraction: changing instruction behavior requires only reprogramming the EEPROM, not rewiring hardware.
- Debugging hardware is fundamentally different from debugging software, manually tracing signals with LEDs and a multimeter requires methodical thinking and patience.
- Team coordination and clearly defined module interfaces are as important as technical skill in a hardware project.
- The 4-bit address space (16 bytes) is the most constraining design limitation, larger programs require architectural expansion.

13. Future Improvements

Improvement	Description	Complexity
PCB version	Transfer breadboard design to printed circuit board for reliability and compactness	High
Expanded RAM	Split instruction fetch into two clock cycles: one for the opcode and one for the operand, allowing a full 8-bit operand address without widening the bus. Replace 74LS189 with 62256 SRAM to support 256-byte address space	Medium
Additional instructions	Implement MUL (multiply), stack operations PUSH/POP, and subroutine CALL/RET	Medium

14. Appendix

Appendix A - Fibonacci Program Listing

The following program computes the Fibonacci sequence and outputs each value to the 7-segment display. It fits entirely within the 16-byte RAM.

Address	Instruction	Binary	Comment
0	LDA 14	0001 1110	Load value 'a' from address 14
1	ADD 15	0010 1111	A = a + b
2	STA 14	0100 1110	Store sum back to address 14
3	OUT	1110 0000	Display result
4	SUB 15	0011 1111	Recover old 'a': (a+b) - b = a
5	STA 15	0100 1111	b = old a
6	JMP 0	0110 0000	Loop forever
14	(data) 1	0000 0001	Initial value of 'a'
15	(data) 1	0000 0001	Initial value of 'b'

Appendix B - Control Signal Reference

Signal	Active	Function
HLT	HIGH	Halt the clock
MI	HIGH	Memory Address Register in (load from bus)
RI	HIGH	RAM in (write bus to current address)
RO	HIGH	RAM out (place RAM value on bus)
IO	HIGH	Instruction Register out (lower 4 bits to bus)
II	HIGH	Instruction Register in
AI	HIGH	A Register in
AO	HIGH	A Register out
EO	HIGH	ALU out (place ALU result on bus)
SU	HIGH	ALU subtract mode
BI	HIGH	B Register in
OI	HIGH	Output Register in (to display)
CE	HIGH	Program Counter enable (increment)
CO	HIGH	Program Counter out
J	HIGH	Jump (load PC from bus)
FI	HIGH	Flags Register in

Appendix C - Team Contact & Roles

Member	Primary Module	Secondary Role
Jana	Clock, Counter, JK Flip-Flop (Videos 2-5, 24-29)	Testing and wiring
Amr	Latches, Registers (Videos 6-13)	Testing and wiring
Abdelrahman	ALU (Videos 14-18)	Layout and build
Monica	RAM Module (Videos 19-23)	Testing and wiring
Hamza	Display, Control Logic, EEPROM (Videos 30-44)	EEPROM programming

End of Document

8-Bit Breadboard Computer | Team Project | 2025-2026